```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("imdevskp/corona-virus-report")

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/imdevskp/corona-virus-report/166.archive...
100%|██████████| 19.0M/19.0M [00:02<00:00, 8.36MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/imdevskp/corona-virus-report/versions/166
```

```
import os

# List the contents of the downloaded dataset directory
print(os.listdir(path))
```

```
['full_grouped.csv', 'day_wise.csv', 'country_wise_latest.csv', 'covid_19_clean_complete.csv', 'usa_county_wise.csv', 'worldomet
```

```
# Download an image dataset suitable for AlexNet, VGG16, VGG19
# The 'Chest X-Ray Images (Pneumonia)' dataset is a good option for image classification.
path = kagglehub.dataset_download("paultimothymooney/chest-xray-pneumonia")

print("Path to image dataset files:", path)
```

```
Using Colab cache for faster access to the 'chest-xray-pneumonia' dataset.
Path to image dataset files: /kaggle/input/chest-xray-pneumonia
```

```
import os

# List the contents of the newly downloaded image dataset directory
print(os.listdir(path))
```

```
['chest_xray']
```

```
import os

# List the contents of the 'chest_xray' subdirectory
print(os.listdir(os.path.join(path, 'chest_xray')))
```

```
['chest_xray', '__MACOSX', 'val', 'test', 'train']
```

```
import os

# Let's inspect the 'train' directory to see the class structure
train_path = os.path.join(path, 'chest_xray', 'train')
print(f"Contents of the 'train' directory: {os.listdir(train_path)}")
```

```
Contents of the 'train' directory: ['PNEUMONIA', 'NORMAL']
```

```
import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# Define data transformations
# All pre-trained models expect input images normalized in the same way,
# i.e. mini-batches of 3-channel RGB images of shape (3 x H x W),
# where H and W are expected to be at least 224.
# The images have to be loaded in to a range of [0, 1] and then normalized
# using mean = [0.485, 0.456, 0.406] and std = [0.229, 0.224, 0.225].

data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)), # Resize images to 224x224 for models
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
```

```
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'test': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

# Define dataset paths
data_dir = os.path.join(path, 'chest_xray')
image_datasets = {
    x: torchvision.datasets.ImageFolder(os.path.join(data_dir, x), data_transforms[x])
    for x in ['train', 'val', 'test']
}

# Define data loaders
dataloaders = {
    x: DataLoader(image_datasets[x], batch_size=32, shuffle=True, num_workers=2) # Using a batch size of 32 for example
    for x in ['train', 'val', 'test']
}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}
class_names = image_datasets['train'].classes

print(f"Dataset sizes: {dataset_sizes}")
print(f"Class names: {class_names}")
```

```
Dataset sizes: {'train': 5216, 'val': 16, 'test': 624}
Class names: ['NORMAL', 'PNEUMONIA']
```

```
import torch.nn as nn
import torchvision.models as models
import torch.optim as optim
import time
import copy

# Set device to GPU if available, else CPU
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# --- VGG16 ---
model_vgg16 = models.vgg16(weights='IMAGENET1K_V1') # Load pre-trained VGG16

# Freeze all parameters in the feature extractor
for param in model_vgg16.features.parameters():
    param.requires_grad = False

# Modify the classifier for our specific task (2 classes: NORMAL, PNEUMONIA)
num_classes = len(class_names) # Should be 2 based on previous output
model_vgg16.classifier[6] = nn.Linear(model_vgg16.classifier[6].in_features, num_classes)

model_vgg16 = model_vgg16.to(device)

criterion_vgg16 = nn.CrossEntropyLoss()
optimizer_vgg16 = optim.Adam(model_vgg16.classifier.parameters(), lr=0.001)

print("VGG16 model loaded and modified successfully.")
```

```
Using device: cuda:0
VGG16 model loaded and modified successfully.
```

```
def train_model(model, criterion, optimizer, scheduler, num_epochs=10):
    since = time.time()

    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1}')
        print('-' * 10)

        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()  # Set model to training mode
```

```
            else:
                model.eval()   # Set model to evaluate mode

            running_loss = 0.0
            running_corrects = 0

            # Iterate over data.
            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                # zero the parameter gradients
                optimizer.zero_grad()

                # forward
                # track history only in train
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    # backward + optimize only if in training phase
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                # statistics
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)
            if phase == 'train':
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double() / dataset_sizes[phase]

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

            # deep copy the model if it's the best accuracy
            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())

        print()

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s')
    print(f'Best val Acc: {best_acc:.4f}')

    # load best model weights
    model.load_state_dict(best_model_wts)
    return model

print("Training function `train_model` defined.")
```

```
Training function `train_model` defined.
```

```
from torch.optim import lr_scheduler

# Setup a learning rate scheduler
# Decay LR by a factor of 0.1 every 7 epochs
scheduler_vgg16 = lr_scheduler.StepLR(optimizer_vgg16, step_size=7, gamma=0.1)

# Train VGG16 model
print("\n--- Starting VGG16 Training ---")
model_vgg16 = train_model(model_vgg16, criterion_vgg16, optimizer_vgg16, scheduler_vgg16, num_epochs=3) # Training for 3 epochs
print("--- VGG16 Training Complete ---")
```

```

--- Starting VGG16 Training ---
Epoch 0/2
----------
train Loss: 0.3732 Acc: 0.9254
val Loss: 0.0219 Acc: 1.0000

Epoch 1/2
----------
train Loss: 0.2270 Acc: 0.9605
val Loss: 0.3264 Acc: 0.8750
```

```
Epoch 2/2
----------
train Loss: 0.3471 Acc: 0.9609
val Loss: 0.0017 Acc: 1.0000

Training complete in 4m 25s
Best val Acc: 1.0000
--- VGG16 Training Complete ---
```

```
# --- VGG19 ---
model_vgg19 = models.vgg19(weights='IMAGENET1K_V1') # Load pre-trained VGG19

# Freeze all parameters in the feature extractor
for param in model_vgg19.features.parameters():
    param.requires_grad = False

# Modify the classifier for our specific task
num_classes = len(class_names)
model_vgg19.classifier[6] = nn.Linear(model_vgg19.classifier[6].in_features, num_classes)

model_vgg19 = model_vgg19.to(device)

criterion_vgg19 = nn.CrossEntropyLoss()
optimizer_vgg19 = optim.Adam(model_vgg19.classifier.parameters(), lr=0.001)
scheduler_vgg19 = lr_scheduler.StepLR(optimizer_vgg19, step_size=7, gamma=0.1)

print("VGG19 model loaded and modified successfully.")
```

```
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.pth
100%|██████████| 548M/548M [00:05<00:00, 110MB/s]
VGG19 model loaded and modified successfully.
```

```
# Train VGG19 model
print("\n--- Starting VGG19 Training ---")
model_vgg19 = train_model(model_vgg19, criterion_vgg19, optimizer_vgg19, scheduler_vgg19, num_epochs=3) # Training for 3 epochs
print("--- VGG19 Training Complete ---")
```

```
--- Starting VGG19 Training ---
Epoch 0/2
----------
train Loss: 0.4489 Acc: 0.9289
val Loss: 0.9522 Acc: 0.8750

Epoch 1/2
----------
train Loss: 0.2627 Acc: 0.9601
val Loss: 0.1275 Acc: 0.9375

Epoch 2/2
----------
train Loss: 0.1448 Acc: 0.9711
val Loss: 0.7474 Acc: 0.8750

Training complete in 4m 28s
Best val Acc: 0.9375
--- VGG19 Training Complete ---
```

```
# --- ResNet ---
# Using ResNet18 as an example, you can choose other ResNet variants like resnet34, resnet50, etc.
model_resnet = models.resnet18(weights='IMAGENET1K_V1') # Load pre-trained ResNet18

# ResNet's classifier is the 'fc' layer. We need to modify it.
num_classes = len(class_names)
num_ftrs = model_resnet.fc.in_features
model_resnet.fc = nn.Linear(num_ftrs, num_classes)

# Move the model to the appropriate device
model_resnet = model_resnet.to(device)

criterion_resnet = nn.CrossEntropyLoss()
optimizer_resnet = optim.Adam(model_resnet.fc.parameters(), lr=0.001)
scheduler_resnet = lr_scheduler.StepLR(optimizer_resnet, step_size=7, gamma=0.1)

print("ResNet model loaded and modified successfully.")
```

```
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd
100%|██████████| 44.7M/44.7M [00:00<00:00, 218MB/s]ResNet model loaded and modified successfully.
```

```
# Train ResNet model
print("\n--- Starting ResNet Training ---")
model_resnet = train_model(model_resnet, criterion_resnet, optimizer_resnet, scheduler_resnet, num_epochs=3) # Training for 3 e
print("--- ResNet Training Complete ---")
```

```
--- Starting ResNet Training ---
Epoch 0/2
----------
train Loss: 0.2432 Acc: 0.8997
val Loss: 0.4080 Acc: 0.6875

Epoch 1/2
----------
train Loss: 0.1655 Acc: 0.9354
val Loss: 0.3655 Acc: 0.7500

Epoch 2/2
----------
train Loss: 0.1367 Acc: 0.9479
val Loss: 0.2511 Acc: 0.8750

Training complete in 3m 59s
Best val Acc: 0.8750
--- ResNet Training Complete ---
```

```
# --- AlexNet ---
model_alexnet = models.alexnet(weights='IMAGENET1K_V1') # Load pre-trained AlexNet

# Modify the classifier for our specific task (2 classes: NORMAL, PNEUMONIA)
num_classes = len(class_names)

# AlexNet's classifier is a sequence of layers. We need to replace the last linear layer.
# The original last layer is model_alexnet.classifier[6] which is an nn.Linear layer.
# We need to replace it with a new nn.Linear layer that outputs 'num_classes' features.
model_alexnet.classifier[6] = nn.Linear(model_alexnet.classifier[6].in_features, num_classes)

# Move the model to the appropriate device
model_alexnet = model_alexnet.to(device)

criterion_alexnet = nn.CrossEntropyLoss()
optimizer_alexnet = optim.Adam(model_alexnet.classifier.parameters(), lr=0.001)
scheduler_alexnet = lr_scheduler.StepLR(optimizer_alexnet, step_size=7, gamma=0.1)

print("AlexNet model loaded and modified successfully.")
```

```
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to /root/.cache/torch/hub/checkpoints/alexnet-owt-7b
100%|██████████| 233M/233M [00:01<00:00, 124MB/s]
AlexNet model loaded and modified successfully.
```

```
# Train AlexNet model
print("\n--- Starting AlexNet Training ---")
model_alexnet = train_model(model_alexnet, criterion_alexnet, optimizer_alexnet, scheduler_alexnet, num_epochs=3) # Training fc
print("--- AlexNet Training Complete ---")
```

```
--- Starting AlexNet Training ---
Epoch 0/2
----------
train Loss: 0.2203 Acc: 0.9400
val Loss: 1.5810 Acc: 0.5625

Epoch 1/2
----------
train Loss: 0.1604 Acc: 0.9553
val Loss: 0.1127 Acc: 0.9375

Epoch 2/2
----------
train Loss: 0.1004 Acc: 0.9711
val Loss: 0.5936 Acc: 0.7500

Training complete in 3m 51s
```

```
Best val Acc: 0.9375
--- AlexNet Training Complete ---
```

```
Start coding or generate with AI.
```

```
# --- EfficientNet (Exceptional Model) ---
# Using EfficientNet_B0 as an example, you can choose other variants like EfficientNet_B1, B2, etc.
model_efficientnet = models.efficientnet_b0(weights='IMAGENET1K_V1') # Load pre-trained EfficientNet_B0

# EfficientNet's classifier is within the `classifier` module, and the last layer is a Linear layer.
num_classes = len(class_names)
# Get the in_features of the original last linear layer in the classifier
num_ftrs_efficientnet = model_efficientnet.classifier[-1].in_features
# Replace the last layer with a new one for our number of classes
model_efficientnet.classifier[-1] = nn.Linear(num_ftrs_efficientnet, num_classes)

# Move the model to the appropriate device
model_efficientnet = model_efficientnet.to(device)

criterion_efficientnet = nn.CrossEntropyLoss()
optimizer_efficientnet = optim.Adam(model_efficientnet.classifier.parameters(), lr=0.001)
scheduler_efficientnet = lr_scheduler.StepLR(optimizer_efficientnet, step_size=7, gamma=0.1)

print("EfficientNet model loaded and modified successfully.")
```

```
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/
100%|██████████| 20.5M/20.5M [00:00<00:00, 219MB/s]EfficientNet model loaded and modified successfully.
```

```
# Train EfficientNet model
print("\n--- Starting EfficientNet Training ---")
model_efficientnet = train_model(model_efficientnet, criterion_efficientnet, optimizer_efficientnet, scheduler_efficientnet, nu
print("--- EfficientNet Training Complete ---")
```

```
--- Starting EfficientNet Training ---
Epoch 0/2
----------
train Loss: 0.2697 Acc: 0.8993
val Loss: 0.4361 Acc: 0.8125

Epoch 1/2
----------
train Loss: 0.1706 Acc: 0.9373
val Loss: 0.3100 Acc: 0.8125

Epoch 2/2
----------
train Loss: 0.1585 Acc: 0.9423
val Loss: 0.3977 Acc: 0.7500

Training complete in 4m 9s
Best val Acc: 0.8125
--- EfficientNet Training Complete ---
```